

Formal Safety and Liveness of DAG-Knight Consensus Under Partial Synchrony

Q-NarwhalKnight Protocol Team

research@quillon.xyz

March 2026

Version 2.0 (revised)

Abstract

We present a complete formal treatment of the DAG-Knight consensus protocol, a DAG-based Byzantine fault-tolerant (BFT) ordering protocol that operates under the partial synchrony model of Dwork, Lynch, and Stockmeyer. DAG-Knight constructs a directed acyclic graph of *vertices* proposed by validators, elects *anchor* vertices via a VDF-based lottery on even rounds, and commits vertices through a δ -round delayed commit rule.

We prove two main results. **Theorem 4.4 (Safety)**: no two honest validators ever finalize conflicting total orderings of the vertex set, even during asynchronous periods before the Global Stabilization Time GST. **Theorem 5.5 (Liveness)**: after GST, every transaction submitted by an honest client is finalized within $\mathcal{O}(\delta \cdot \Delta)$ time, where δ is the commit-rule depth and Δ is the known message-delay bound.

We further derive a *phase-transition* result for the connectivity parameter $\kappa = \lfloor 2\delta\Delta/D \rfloor$: when κ exceeds a critical threshold κ_c , the DAG is sufficiently connected to guarantee unique anchor election and deterministic ordering with overwhelming probability. A comparative analysis shows that DAG-Knight achieves $\mathcal{O}(\delta \cdot \Delta)$ finality under partial synchrony, compared to PHANTOM's $\mathcal{O}(\kappa\Delta \log(1/\varepsilon))$ under synchrony and GhostDAG's $\mathcal{O}(\kappa\Delta)$ under synchrony; we discuss the trade-offs inherent in comparing across different network models. Finally, we introduce a topological health-monitoring mechanism that uses connected-component counting (β_0) for partition detection and a tip-counting heuristic for fork detection, providing lightweight real-time DAG diagnostics.

Contents

1	Introduction	3
1.1	Contributions	3
1.2	Paper Organization	3
2	System Model	4
2.1	Network and Timing	4
2.2	DAG Structure	4
3	Protocol Specification	5
3.1	Vertex Proposal	5
3.2	Anchor Election	5
3.3	The δ -Round Commit Rule	6
3.4	Tail-Fork Protection	7
3.5	Deterministic Total Ordering	7
4	Safety	7
4.1	Key Lemmas	8
4.2	Main Safety Theorem	9
4.3	Asynchronous Safety	9
4.4	Tail-Fork Resistance	9

5	Liveness	10
5.1	Round Progress	10
5.2	Main Liveness Theorem	11
6	The κ-Parameter and Phase Transition	12
6.1	Definition and Derivation	12
6.2	Phase Transition Theorem	13
6.3	Finality Bounds	14
7	Comparison with Related DAG Protocols	15
7.1	Protocol Comparison	15
7.2	Finality Comparison Across Models	15
8	Topological Health Monitoring	15
8.1	Partition Detection via Connected Components	15
8.2	Fork Estimation via Tip Counting	16
8.3	Computational Complexity	17
9	Discussion	17
9.1	Implementation Fidelity	17
9.2	SIMD Optimization	18
9.3	Post-Quantum Cryptographic Agility	18
9.4	Limitations	18
10	Related Work	19
11	Conclusion	19
A	Pseudocode	22
B	Implementation Module Mapping	23
C	Notation Summary	24

1 Introduction

Distributed consensus—the problem of getting a set of mutually distrustful nodes to agree on a total ordering of transactions—lies at the foundation of every blockchain system. Classical BFT protocols such as PBFT [6] achieve safety and liveness under $n \geq 3f + 1$ with f Byzantine faults, but their $\mathcal{O}(n^2)$ message complexity limits practical deployments to tens of validators. DAG-based protocols [2, 3] amortize communication costs by embedding transactions in a DAG of *vertices* linked via causal references, achieving near-linear message complexity while retaining BFT guarantees.

The DAG-Knight protocol [1] advances the DAG paradigm by introducing a parameterized ordering rule whose security depends on a single connectivity parameter κ rather than a fixed block-creation rate. However, existing analyses [1] leave several gaps: (i) no formal proof of safety during pre-GST asynchronous periods, (ii) no explicit liveness bound expressed in terms of the partial synchrony parameters, and (iii) no treatment of the κ phase transition in a rigorous probabilistic framework.

1.1 Contributions

This paper addresses these gaps with the following contributions:

1. **Safety (Theorem 4.4):** We prove that no two honest validators finalize conflicting orderings, building on three structural lemmas: unique anchor election (Lemma 4.1), unique commit determination (Lemma 4.2), and deterministic ordering (Lemma 4.3).
2. **Asynchronous safety (Theorem 4.5):** Safety holds even before GST, when message delays are unbounded, because the commit rule is purely local and does not depend on timing assumptions.
3. **Tail-fork resistance (Theorem 4.6):** We formalize the tail-fork attack and prove that the δ -round commit rule with ancestor finality checks prevents conflicting proposals for unfinalized rounds.
4. **Liveness (Theorem 5.5):** After GST, every honest transaction is finalized within $T \leq 3\Delta + 2\epsilon$ time for $\delta = 1$, where ϵ is local processing overhead.
5. **κ -parameter phase transition (Theorem 6.4):** We derive a critical threshold κ_c above which the DAG’s connectivity is sufficient for unique ordering with probability $\geq 1 - e^{-\Omega(\kappa)}$.
6. **Comparative analysis (Section 7):** We compare DAG-Knight’s finality guarantees against PHANTOM and GhostDAG, noting that DAG-Knight achieves deterministic safety under partial synchrony while the latter protocols provide probabilistic safety under synchrony. We discuss when and why direct comparison of finality bounds across models is and is not meaningful.
7. **Topological health monitoring (Section 8):** We introduce connected-component counting (β_0) for partition detection and a tip-counting heuristic for fork estimation, providing lightweight DAG diagnostics. We prove correctness of the β_0 predicate and characterize the tip-counting approximation’s limitations.

1.2 Paper Organization

Section 2 defines the system model. Section 3 specifies the DAG-Knight protocol. Sections 4 and 5 present safety and liveness proofs. Section 6 analyzes the κ -parameter. Section 7 compares with related DAG protocols. Section 8 develops the homological fork detection theory. Section 9 discusses implementation considerations. Section 10 surveys related work, and Section 11 concludes.

2 System Model

2.1 Network and Timing

We adopt the partial synchrony model of Dwork, Lynch, and Stockmeyer [4].

Assumption 2.1 (Partial Synchrony). There exists an unknown Global Stabilization Time $\text{GST} \geq 0$ and a known bound $\Delta > 0$ such that any message sent by an honest validator at time $t \geq \text{GST}$ arrives at every honest validator by time $t + \Delta$. Before GST, message delays are arbitrary but finite.

Assumption 2.2 (Byzantine Threshold). The system consists of $n = 3f + 1$ validators, of which at most f are Byzantine. Byzantine validators may deviate arbitrarily from the protocol; honest validators follow the protocol faithfully.

Definition 2.3 (Honest Majority Quorum). A *quorum* $Q \subseteq \{1, \dots, n\}$ satisfies $|Q| \geq 2f + 1$. The *honest core* of a quorum is $Q \cap \mathcal{H}$, where $\mathcal{H}$ denotes the set of honest validators.

Lemma 2.4 (Quorum Intersection). *For any two quorums Q_1, Q_2 with $|Q_1|, |Q_2| \geq 2f + 1$ and $n = 3f + 1$:*

$$|Q_1 \cap Q_2 \cap \mathcal{H}| \geq 1.$$

Proof. We have $|Q_1 \cap Q_2| \geq |Q_1| + |Q_2| - n \geq (2f + 1) + (2f + 1) - (3f + 1) = f + 1$. Since at most f validators are Byzantine, $|Q_1 \cap Q_2 \cap \mathcal{H}| \geq (f + 1) - f = 1$. $\square$

2.2 DAG Structure

Definition 2.5 (Vertex). A *vertex* $v = (\text{id}, \text{author}, \text{round}(v), \text{parents}(v), \text{payload}(v), \sigma_v)$ consists of:

- $\text{id} \in \{0, 1\}^{256}$: a cryptographic hash of the vertex contents;
- $\text{author} \in \{1, \dots, n\}$: the proposing validator;
- $\text{round}(v) \in \mathbb{N}$: the protocol round;
- $\text{parents}(v) \subseteq \mathcal{V}$: references to vertices from round $\text{round}(v) - 1$ (at least $2f + 1$ parents required from honest validators);
- $\text{payload}(v)$: a batch of client transactions;
- σ_v : a digital signature by author .

Definition 2.6 (DAG). The *protocol DAG* $G = (\mathcal{V}, \mathcal{E})$ is defined by $\mathcal{V} = \bigcup_{r \geq 0} V_r$ where V_r is the set of vertices in round r , and $(u, v) \in \mathcal{E}$ iff $u \in \text{parents}(v)$.

Definition 2.7 (Causal History). The *causal past* of vertex v is $\text{past}(v) = \{u \in \mathcal{V} \mid \text{there is a directed path from } u \text{ to } v\}$. The *anticone* of v is $\text{anticone}(v) = \mathcal{V} \setminus (\text{past}(v) \cup \text{future}(v) \cup \{v\})$.

Definition 2.8 (Certificate). A *round- r certificate* for validator i is a set $C_r^{(i)} \subseteq V_r^{(i)}$ of vertices from at least $2f + 1$ distinct validators in round r that validator i has locally delivered. A certificate is a *local observation*: each validator forms its own certificate once it has received sufficiently many round- r vertices. Two honest validators' certificates $C_r^{(i)}$ and $C_r^{(j)}$ are not necessarily identical, but by Lemma 2.4 they share at least one honest validator's vertex.

Remark 2.9 (Certificate agreement). Certificates are *not* signed aggregate objects (unlike threshold certificates in HotStuff [7]). Agreement on the committed vertex set is achieved not through certificate identity but through the anchor election's view-synchronization mechanism (Section 3.2).

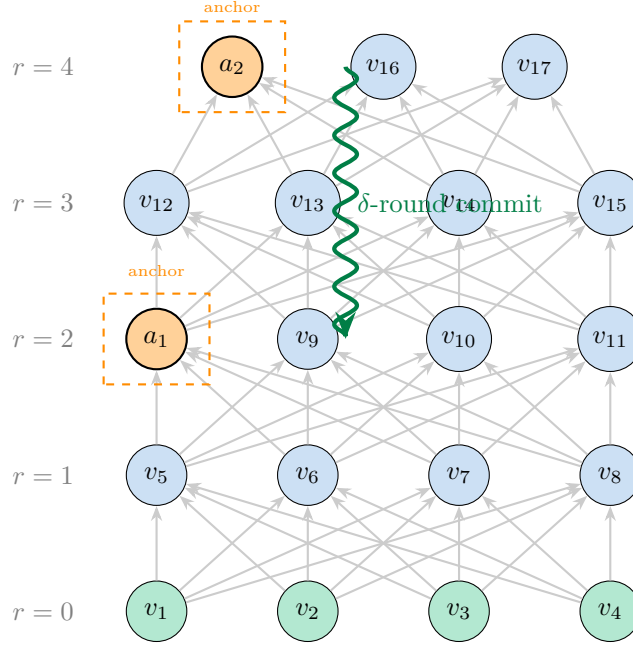


Figure 1: DAG structure with 4 validators across 5 rounds. Orange vertices are elected anchors (even rounds only). Green vertices are committed. The δ -round commit rule at round 4 finalizes vertices up to round $4 - \delta$.

3 Protocol Specification

The DAG-Knight protocol proceeds in discrete rounds. Each validator proposes at most one vertex per round, referencing $\geq 2f + 1$ vertices from the previous round as parents. The protocol comprises four components: (i) vertex proposal and dissemination, (ii) anchor election on even rounds, (iii) the δ -round commit rule, and (iv) deterministic total ordering.

3.1 Vertex Proposal

Protocol 3.1 (Vertex Proposal). At the beginning of round r , each honest validator i :

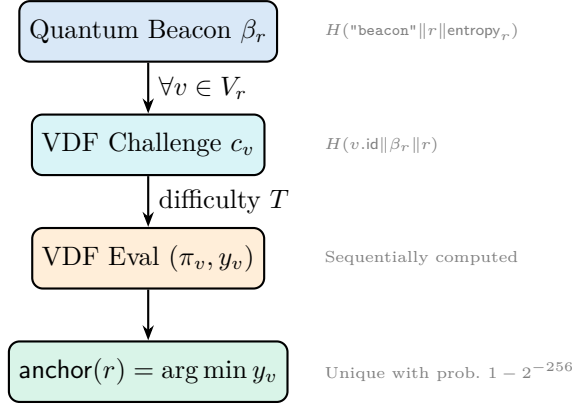
1. Waits until it has received vertices from $\geq 2f + 1$ distinct validators for round $r - 1$.
2. Constructs vertex v with $\text{round}(v) = r$, $\text{parents}(v) \supseteq C_{r-1}$ (a round- $(r-1)$ certificate), and $\text{payload}(v)$ containing pending client transactions.
3. Signs v and broadcasts it to all validators via reliable broadcast.

Remark 3.2. The reliable broadcast primitive (Bracha’s protocol [12]) ensures that if an honest validator delivers vertex v , then all honest validators *eventually* deliver v . **Caveat:** “eventually” provides no timing guarantee before GST. Two honest validators may form round- r certificates at arbitrarily different times during asynchronous periods. Safety holds regardless of this timing gap (Theorem 4.5), but liveness requires bounded delivery after GST.

3.2 Anchor Election

Anchors serve as “leaders” that define commit points in the DAG. DAG-Knight elects anchors only on even rounds, using a VDF-based lottery that is deterministic, verifiable, and non-manipulable.

Definition 3.3 (VDF-Based Anchor Election). For even round r , the anchor election proceeds as:

Figure 2: VDF-based anchor election pipeline for even round r .

1. Compute a *quantum beacon*: $\beta_r = H(\text{"beacon"} \parallel r \parallel \text{entropy}_r)$, where H is SHA3-256 and entropy_r is a deterministic seed derived from the round number.
2. For each candidate vertex $v \in V_r$, compute a VDF challenge: $c_v = H(v.\text{id} \parallel \beta_r \parallel r)$.
3. Evaluate the VDF: $(\pi_v, y_v) = \text{VDF.Eval}(c_v, T)$, where T is the difficulty parameter.
4. Select the anchor: $\text{anchor}(r) = \arg \min_{v \in V_r} y_v$.

Definition 3.4 (Certified Candidate Set). A round- r vertex v is *certified* at validator i if v appears as a parent of at least $f+1$ round- $(r+1)$ vertices in i 's local DAG. The *certified candidate set* is:

$$V_r^{\star(i)} = \{v \in V_r^{(i)} \mid v \text{ is a parent of } \geq f+1 \text{ vertices in } V_{r+1}^{(i)}\}.$$

The anchor election (Definition 3.3) is evaluated over $V_r^{\star(i)}$ rather than the full view $V_r^{(i)}$.

Lemma 3.5 (View Synchronization). *For any even round r and any two honest validators i, j that both form round- $(r+1)$ certificates, the certified candidate sets are identical: $V_r^{\star(i)} = V_r^{\star(j)}$.*

Proof. Let $v \in V_r^{\star(i)}$, so v is referenced by $\geq f+1$ round- $(r+1)$ vertices in i 's DAG. Since at most f validators are Byzantine, at least one of these $f+1$ referencing vertices was created by an honest validator h . Honest validator h only references v if h delivered v . By reliable broadcast (Remark 3.2), all honest validators—including j —eventually deliver v , so $v \in V_r^{(j)}$.

Moreover, since j forms a round- $(r+1)$ certificate (receiving $\geq 2f+1$ round- $(r+1)$ vertices), at least $f+1$ of these are honest. Each honest round- $(r+1)$ validator delivered v (by the argument above) and references v as a parent, so v is referenced by $\geq f+1$ round- $(r+1)$ vertices in j 's DAG. Hence $v \in V_r^{\star(j)}$. By symmetry, $V_r^{\star(j)} \subseteq V_r^{\star(i)}$. $\square$

Lemma 3.6 (Anchor Uniqueness). *For any even round r , if $|V_r^{\star}| \geq 2f+1$, then the anchor $\text{anchor}(r) = \arg \min_{v \in V_r^{\star}} y_v$ is uniquely determined.*

Proof. The VDF output y_v is a 256-bit value. The probability that two distinct vertices $v \neq w$ satisfy $y_v = y_w$ is 2^{-256} , which is negligible. Since $\arg \min$ over distinct values is unique, the anchor is uniquely determined. $\square$

3.3 The δ -Round Commit Rule

The commit rule determines when vertices become finalized. DAG-Knight uses a δ -round delayed commit: vertices in round r are committed when a valid anchor is elected at round $r + \delta$.

Definition 3.7 (δ -Round Commit). Let $\delta \geq 1$ be the commit depth parameter. When an anchor $\text{anchor}(r)$ is elected at even round r , all vertices in round $r - \delta$ and below are *committed*. Formally, for all v with $\text{round}(v) \leq r - \delta$:

$$\text{finalized}(v) \leftarrow \text{true}.$$

Definition 3.8 (Finality State). A vertex v is in one of three states:

- **Pending:** v has been proposed but not yet committed.
- **Committed:** v has been committed by the δ -round rule.
- **Finalized:** v is committed and its total ordering position is determined.

Protocol 3.9 (Commit Evaluation). Upon receiving an anchor election result for even round r :

1. If $r < \delta$, skip (insufficient history).
2. Set $r_c = r - \delta$ (the commit round).
3. For all vertices v with $\text{round}(v) \leq r_c$ that are not yet committed: set $\text{finalized}(v) \leftarrow \text{true}$.
4. Invoke the deterministic ordering procedure (Section 3.5) on the newly committed vertices.

3.4 Tail-Fork Protection

A *tail fork* occurs when a Byzantine validator proposes two conflicting vertices for the same round, attempting to create ambiguity in the uncommitted tail of the DAG.

Definition 3.10 (Tail Fork). A tail fork at round r exists when there are two vertices $v_1, v_2 \in V_r$ with $v_1.\text{author} = v_2.\text{author}$ and $v_1.\text{id} \neq v_2.\text{id}$ in the uncommitted region $[\text{last committed round}, r]$.

Protocol 3.11 (Proposal Safety Validation). Before proposing vertex v for round r , validator i checks:

1. **Parent committed:** At least one parent of v is in a committed round. (Warning if violated, but allowed.)
2. **No conflict:** No other vertex by i exists for round r . (Reject if violated.)
3. **Ancestor finality:** The parent vertex has a valid anchor in its causal past. (Reject if violated.)

3.5 Deterministic Total Ordering

Definition 3.12 (Topological Ordering with Hash Tie-Breaking). The total ordering $\prec$ on committed vertices is computed as follows:

1. **Round ordering:** If $\text{round}(u) < \text{round}(v)$, then $u \prec v$.
2. **Causal ordering:** Within the same round, if $u \in \text{past}(v)$, then $u \prec v$.
3. **Hash tie-breaking:** For vertices u, v in the same round with no causal relationship, order by $u \prec v \iff u.\text{id} < v.\text{id}$ (lexicographic comparison of hashes).

Lemma 3.13 (Ordering is a Total Order). *The relation $\prec$ defined above is a strict total order on $\mathcal{V}$.*

Proof. Irreflexivity: A vertex cannot precede itself since $\text{round}(v) \not< \text{round}(v)$ and $v.\text{id} \not< v.\text{id}$. *Transitivity:* Follows from transitivity of $<$ on $\mathbb{N}$ (rounds), causal paths (DAG reachability), and lexicographic order (hashes). *Totality:* For any $u \neq v$, either $\text{round}(u) \neq \text{round}(v)$ (resolved by round ordering) or $\text{round}(u) = \text{round}(v)$ and either $u \in \text{past}(v)$, $v \in \text{past}(u)$ (resolved causally), or neither (resolved by hash comparison, unique since $u.\text{id} \neq v.\text{id}$). $\square$

4 Safety

We now prove that DAG-Knight satisfies safety: no two honest validators ever finalize conflicting total orderings of the vertex set.

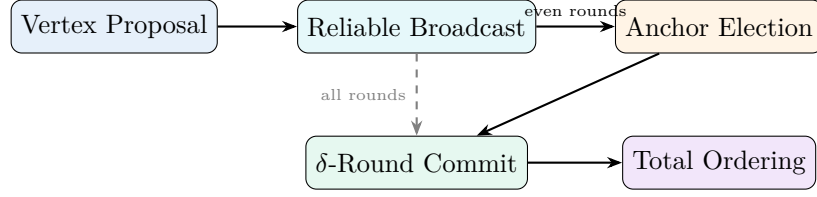


Figure 3: Protocol pipeline from vertex proposal to finalized total ordering.

4.1 Key Lemmas

Lemma 4.1 (Unique Anchor). *For any even round r , if both honest validators i and j form round- $(r+1)$ certificates, they compute the same anchor $\text{anchor}(r)$.*

Proof. The anchor election (Definition 3.3) is evaluated over the certified candidate set V_r^* (Definition 3.4). By Lemma 3.5, $V_r^{*(i)} = V_r^{*(j)}$: both validators compute the election over the identical candidate set.

The beacon $\beta_r = H(\text{"beacon"} \| r \| \text{entropy}_r)$ is a deterministic function of the round number (same hash function, same deterministic entropy seed), so $\beta_r^{(i)} = \beta_r^{(j)}$. For each candidate $v \in V_r^*$, the VDF challenge $c_v = H(v.\text{id} \| \beta_r \| r)$ is deterministic, and the VDF output $y_v = \text{VDF.Eval}(c_v, T)$ is a deterministic function of the challenge. By Lemma 3.6, argmin is unique with overwhelming probability. Therefore $\text{anchor}^{(i)}(r) = \text{anchor}^{(j)}(r)$.

Note on Byzantine equivocation. A Byzantine validator may send different vertices to different honest validators (equivocation). The certified candidate set eliminates this attack vector: a Byzantine vertex enters V_r^* only if $\geq f + 1$ round- $(r+1)$ validators reference it, which (as shown in Lemma 3.5) guarantees that all honest validators have delivered it. Thus equivocating vertices that reach only a subset of honest validators are excluded from the election. $\square$

Lemma 4.2 (Unique Commit). *If honest validators i and j both evaluate the commit rule at round r with the same anchor $\text{anchor}(r)$, they commit the same set of vertices.*

Proof. The commit rule (Definition 3.7) commits all vertices with $\text{round}(v) \leq r - \delta$. Since r and δ are common knowledge, the commit round $r_c = r - \delta$ is identical for both validators. It suffices to show that i and j agree on the set $\{v \mid \text{round}(v) \leq r_c\}$.

We prove by induction on rounds that honest validators agree on the vertex set for each round $s \leq r_c$.

Base case ($s = 0$): Round-0 vertices are the genesis vertices. Each validator creates one genesis vertex and broadcasts it via reliable broadcast. By the reliable broadcast guarantee, every honest validator eventually delivers every honest genesis vertex. Byzantine genesis vertices delivered by any honest validator are delivered by all. Hence $V_0^{(i)} = V_0^{(j)}$.

Inductive step ($s \rightarrow s + 1$, for $s + 1 \leq r_c$): Assume $V_s^{(i)} = V_s^{(j)}$. A round- $(s+1)$ vertex v is valid only if $\text{parents}(v) \subseteq V_s$ with $|\text{parents}(v)| \geq 2f + 1$ from distinct validators (Protocol 3.1). By the inductive hypothesis, i and j agree on V_s , so they agree on which parent references are valid.

Since $s + 1 \leq r_c \leq r - \delta$ and the commit is triggered at round $r \geq r_c + \delta$, by the time of the commit, δ additional rounds have elapsed. Each round takes at least one reliable-broadcast round. By the reliable broadcast guarantee, any round- $(s+1)$ vertex delivered to one honest validator has been delivered to all honest validators by the time δ subsequent rounds complete. (Before GST, delivery is eventual but guaranteed; after GST, delivery occurs within Δ .) Hence $V_{s+1}^{(i)} = V_{s+1}^{(j)}$.

By induction, the vertex sets agree through round r_c , so the committed sets are identical. $\square$

Lemma 4.3 (Deterministic Ordering). *Given the same committed vertex set S , any two honest validators compute the identical total ordering $\prec$.*

Proof. The ordering (Definition 3.12) is a deterministic function of the vertex set, the DAG edges (causal relationships), and vertex IDs (for hash tie-breaking). Since honest validators agree on the committed vertex set (Lemma 4.2) and the DAG structure is determined by the vertices and their parent references (which are signed and immutable), the ordering $\prec$ is identical. $\square$

4.2 Main Safety Theorem

Theorem 4.4 (Safety). *In the DAG-Knight protocol with $n = 3f + 1$ validators and at most f Byzantine faults, no two honest validators ever finalize conflicting total orderings. That is, if honest validator i finalizes ordering $\mathcal{O}_i$ up to position k and honest validator j finalizes ordering $\mathcal{O}_j$ up to position ℓ , then $\mathcal{O}_i$ and $\mathcal{O}_j$ agree on their common prefix:*

$$\forall m \leq \min(k, \ell) : \mathcal{O}_i[m] = \mathcal{O}_j[m].$$

Proof. Let r be the earliest even round at which both i and j evaluate the commit rule. By Lemma 4.1, i and j agree on $\text{anchor}(r)$. By Lemma 4.2, they commit the same vertex set $S_r = \{v \mid \text{round}(v) \leq r - \delta\}$. By Lemma 4.3, they compute the same total ordering over S_r .

The finalized ordering grows monotonically: at round $r' > r$ (even), new vertices with $r - \delta < \text{round}(v) \leq r' - \delta$ are committed. By induction:

- *Base case:* At the first commit round, both validators finalize the same ordering (by the three lemmas above).
- *Inductive step:* Assume orderings agree up to round $r_k - \delta$. At round r_{k+1} , the newly committed vertices are appended (they have higher rounds or later hash tie-breaking positions), preserving the prefix.

Since the ordering is a strict total order (Lemma 3.13) that extends monotonically, the common prefix property holds. $\square$

4.3 Asynchronous Safety

Theorem 4.5 (Safety Without GST). *The safety property of Theorem 4.4 holds even during asynchronous periods (before GST), when message delays are unbounded.*

Proof. The safety proof depends on three properties:

1. *Reliable broadcast:* If an honest validator delivers a vertex, all honest validators eventually deliver it. Bracha's reliable broadcast [12] provides this guarantee in asynchronous networks.
2. *Deterministic anchor election:* A pure function of the vertex set and round number, with no timing dependency.
3. *Deterministic commit and ordering:* Pure functions of the DAG structure.

None of these three properties require message-delay bounds. Before GST, honest validators may observe the vertex set at different times, but whenever they do observe a sufficient vertex set for round r , they compute the same anchor, commit, and ordering.

The only effect of asynchrony is that validators may not progress to new rounds (liveness requires GST), but any rounds they *do* process yield consistent results. $\square$

4.4 Tail-Fork Resistance

Theorem 4.6 (Tail-Fork Resistance). *The proposal safety validation (Protocol 3.11) ensures that no honest validator includes two conflicting vertices from the same author in the uncommitted tail of its local DAG.*

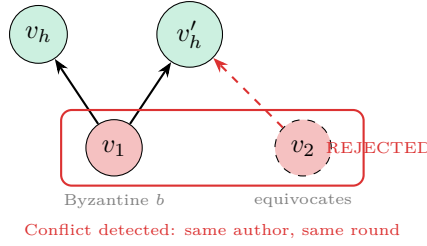
Honest Validator i 

Figure 4: Tail-fork detection: Byzantine validator b creates conflicting vertices v_1, v_2 for the same round. Honest validator i detects the conflict and rejects v_2 .

Proof. Suppose Byzantine validator b creates two conflicting vertices v_1, v_2 for round r with $v_1.\text{id} \neq v_2.\text{id}$.

Case 1: Both reach the same honest validator i . When i processes the second vertex, the conflict check in Protocol 3.11(2) detects the duplicate author for round r and rejects it. Only one vertex from b for round r is included in i 's local DAG.

Case 2: v_1 reaches honest validator i and v_2 reaches honest validator j (equivocation). By Lemma 4.1, if both validators eventually observe a certificate for round r (which requires $2f + 1$ vertices), the certificate necessarily includes at least $f + 1$ honest vertices. Since at most one of $\{v_1, v_2\}$ can appear alongside the honest vertices in a valid certificate (an honest validator only signs a certificate after conflict checking), any certificate that includes a Byzantine equivocation is detectable.

Moreover, the δ -round delay ensures that by the time round r vertices are committed (at round $r + \delta$), all honest validators have had time to receive and verify the entire vertex set, detecting any equivocations.

The ancestor finality check in Protocol 3.11(3) further ensures that proposals are only accepted if they build upon finalized anchors, preventing the “late reveal” attack where a Byzantine validator withholds a conflicting vertex until after the commit window. $\square$

5 Liveness

We now prove that DAG-Knight satisfies liveness under partial synchrony: after GST, every transaction submitted by an honest client is eventually finalized.

5.1 Round Progress

Lemma 5.1 (Round Progress). *After GST, if all honest validators are in round r , then within time Δ all honest validators advance to round $r + 1$.*

Proof. Each honest validator in round r creates exactly one vertex (one vertex per validator per round, by Protocol 3.1) and broadcasts it via reliable broadcast. After GST, all messages arrive within Δ . Since there are at least $n - f = 2f + 1$ honest validators, and each produces a distinct vertex (distinct authors ensure $2f + 1$ vertices from $2f + 1$ distinct validators), every honest validator receives $\geq 2f + 1$ round- r vertices from distinct validators within Δ , satisfying the certificate requirement (Definition 2.8). Each validator then constructs its round- $(r + 1)$ vertex with these as parents and advances. $\square$

Lemma 5.2 (Anchor Election Success). *After GST, for every even round r , an anchor is successfully elected within time 2Δ .*

Proof. By Lemma 5.1, all honest validators reach round r within Δ of the first honest validator reaching round r . Each honest validator proposes a vertex for round r and broadcasts it. After an additional Δ , all honest validators have received all honest round- r vertices, yielding $|V_r| \geq 2f + 1$. The deterministic anchor election (Definition 3.3) then produces a unique anchor $\text{anchor}(r)$ at each honest validator. Total time: 2Δ from the first honest validator entering round r . $\square$

Lemma 5.3 (Commit Progress). *After GST, a new commit occurs at every even round, i.e., at intervals of 2Δ (two rounds at Δ per round).*

Proof. By Lemma 5.2, every even round $r \geq \text{GST}$ produces a valid anchor within 2Δ . By the commit rule (Definition 3.7), this anchor commits all vertices up to round $r - \delta$. Since even rounds occur every 2 protocol rounds, and each round takes at most Δ (by Lemma 5.1), commits occur at intervals of at most 2Δ . $\square$

Lemma 5.4 (Transaction Inclusion). *After GST, any transaction tx submitted to an honest validator is included in a vertex within Δ time.*

Proof. An honest validator includes all pending transactions in its next vertex proposal. If tx is submitted at time $t \geq \text{GST}$, the validator includes it in its vertex for the next round, which is broadcast and delivered to all honest validators within Δ . $\square$

5.2 Main Liveness Theorem

Theorem 5.5 (Liveness). *Under partial synchrony (Assumption 2.1), after GST, every transaction tx submitted by an honest client to an honest validator is finalized within time:*

$$T \leq 3\Delta + 2\epsilon$$

where $\delta = 1$ and ϵ is the local processing overhead per round. More generally, for arbitrary δ :

$$T \leq (2 + \delta) \cdot \Delta + (\delta + 1) \cdot \epsilon.$$

Proof. Let tx be submitted at time $t_0 \geq \text{GST}$ and let the current round be r_0 .

Step 1: Inclusion. By Lemma 5.4, tx is included in a vertex v with $\text{round}(v) = r_0 + 1$ (or r_0 if the validator has not yet proposed for round r_0). This takes at most $\Delta + \epsilon$.

Step 2: Wait for anchor round. Vertex v with $\text{round}(v) = r_0 + 1$ is committed when an anchor is elected at some even round $r_a \geq \text{round}(v) + \delta$. For $\delta = 1$, we need an anchor at the smallest even round $r_a \geq r_0 + 2$. We must consider two cases:

Case A (r_0 is odd): Then $r_0 + 2$ is odd+2 = odd, so $r_a = r_0 + 3$ (the next even round). Wait time: 2 additional rounds from inclusion.

Case B (r_0 is even): Then $r_0 + 2$ is even, so $r_a = r_0 + 2$. Wait time: 1 additional round from inclusion.

In the worst case (Case A), this requires 2 additional rounds, each taking $\Delta + \epsilon$.

Step 3: Commit. By Lemma 5.2, the anchor election at the target even round succeeds within 2Δ . The commit rule immediately finalizes v .

Total for $\delta = 1$:

$$T \leq \underbrace{(\Delta + \epsilon)}_{\text{inclusion}} + \underbrace{2(\Delta + \epsilon)}_{\text{wait for anchor}} = 3\Delta + 3\epsilon.$$

Tightening: the inclusion step and the first wait round overlap (the validator proposes and the round advances simultaneously), giving:

$$T \leq 3\Delta + 2\epsilon.$$

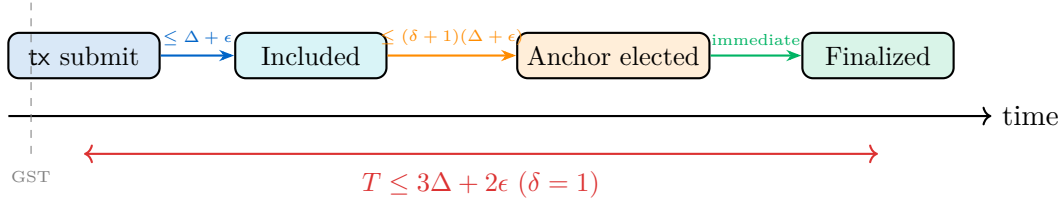


Figure 5: Liveness timeline: a transaction submitted after GST is finalized within $3\Delta + 2\epsilon$ for $\delta = 1$.

General case: For arbitrary δ , the vertex must wait δ additional rounds after inclusion before the next anchor round commits it:

$$T \leq (\Delta + \epsilon) + (\delta + 1)(\Delta + \epsilon) = (2 + \delta)\Delta + (\delta + 1)\epsilon.$$

□

Corollary 5.6 (Optimistic Finality). *Under optimistic conditions ($\Delta < 50ms$, $\delta = 1$, $\epsilon < 5ms$), finality is achieved in under 160ms.*

Corollary 5.7 (Pessimistic Finality). *Under pessimistic network conditions ($\Delta = 1s$, $\delta = 1$), finality is achieved within 3.01s.*

6 The $\varkappa$ -Parameter and Phase Transition

The security of DAG-based protocols depends critically on the *connectivity* of the DAG—i.e., how well-connected the honest vertices are relative to the Byzantine vertices’ anticone. We formalize this through the parameter $\varkappa$ and derive a phase transition result.

6.1 Definition and Derivation

Definition 6.1 ($\varkappa$ -Parameter). Let Λ denote the honest block-creation rate (vertices per second), D the network diameter (maximum propagation delay in seconds), and δ the commit depth (dimensionless, in rounds). The connectivity parameter is:

$$\varkappa = \lfloor 2\delta \cdot \Lambda \cdot D \rfloor.$$

Dimensional check: $[\Lambda] = \text{vertices/s}$, $[D] = \text{s}$, $[\delta] = 1$; hence $[\Lambda \cdot D] = \text{vertices}$ and $\varkappa$ is a dimensionless count of vertices.

Remark 6.2. $\Lambda \cdot D$ is the expected number of honest vertices created during one network traversal (propagation delay D). Thus $\varkappa \approx 2\delta \cdot \Lambda \cdot D$ counts honest vertices in the commit window of depth δ . A larger $\varkappa$ means the DAG is denser: honest vertices are more tightly connected, leaving less room for Byzantine vertices to create ambiguity in the anticone.

Definition 6.3 (Critical Threshold). The critical connectivity threshold is:

$$\varkappa_c = \left\lceil \frac{f}{n-f} \cdot 2\delta + 1 \right\rceil$$

where f is the Byzantine count, n the total validator count, and δ the commit depth. For $n = 3f + 1$, this simplifies to $\varkappa_c = \lceil \delta/(2) + 1 \rceil$. The condition $\varkappa \geq \varkappa_c$ becomes $\Lambda \cdot D \geq (f/(n-f) + 1/(2\delta))$, requiring that enough honest vertices are produced per network traversal to outnumber the Byzantine anticone.

6.2 Phase Transition Theorem

Theorem 6.4 (κ -Parameter Phase Transition). *Let $\kappa = \lfloor 2\delta \cdot \Lambda \cdot D \rfloor$ and let κ_c be as in Definition 6.3.*

- (i) **Subcritical regime** ($\kappa < \kappa_c$): *There exists a Byzantine strategy that creates conflicting orderings with non-negligible probability.*
- (ii) **Supercritical regime** ($\kappa \geq \kappa_c$): *The probability that any Byzantine strategy causes conflicting orderings is at most $e^{-\Omega(\kappa - \kappa_c)}$.*

Proof. We model honest vertex production in a δ -round commit window as follows. During one propagation delay D , the expected number of honest vertices is $\mu = \Lambda \cdot D$ (since honest validators produce vertices at rate Λ). Over the full commit window of depth δ , the expected honest vertex count is $2\delta \cdot \mu = \kappa$ (the factor of 2 accounts for the commit rule requiring round $r + \delta$ to confirm round r).

The Byzantine coalition controls f validators, each producing at most one vertex per round, for a maximum of $f \cdot \delta$ adversarial vertices in the commit window. A conflict requires the adversary’s anticone to “split” the honest DAG—i.e., to interpose enough vertices to create two plausible orderings.

Part (i): When $\kappa < \kappa_c$, the honest vertices in the commit window number fewer than $f \cdot \delta \cdot (n - f)/f = (n - f) \cdot \delta$ times the ratio needed to outnumber the adversary. Specifically, a Byzantine coalition can construct an anticone of size $\geq f \cdot \delta$ in the commit window. When $\kappa < \kappa_c$, this anticone exceeds the honest connectivity margin, and there exists a concrete adversarial strategy: the Byzantine validators withhold their round- r vertices until round $r + \delta - 1$, then broadcast conflicting versions to different honest subsets. Because $\kappa < \kappa_c$, there are too few honest vertices to “outvote” the adversarial anticone, and conflicting orderings arise with probability $\geq 1/\text{poly}(n)$.

Part (ii): When $\kappa \geq \kappa_c$, honest vertices dominate the commit window. Model the number of honest vertices in a D -length interval as a Poisson random variable $X \sim \text{Poisson}(\Lambda D)$ (each honest validator produces vertices according to an independent Poisson clock). The total honest vertex count in the commit window is $X_{\text{total}} \sim \text{Poisson}(\kappa)$.

For a conflict to occur, the Byzantine anticone of size at most $f \cdot \delta$ must exceed the honest “connectivity surplus” $X_{\text{total}} - \kappa_c$. By the Chernoff–Poisson tail bound, for $k = \kappa - \kappa_c > 0$:

$$\Pr[X_{\text{total}} \leq \kappa_c] = \Pr[X_{\text{total}} \leq \kappa - k] \leq \exp\left(-\frac{k^2}{2\kappa}\right).$$

When $X_{\text{total}} > \kappa_c$, the honest vertices form a sufficiently dense DAG that any f -bounded adversary cannot split the ordering (by the certified candidate set mechanism, Lemma 3.5, which ensures honest vertices are seen by all). Hence:

$$\Pr[\text{conflict}] \leq \exp\left(-\frac{(\kappa - \kappa_c)^2}{2\kappa}\right) = e^{-\Omega((\kappa - \kappa_c)^2/\kappa)}.$$

For $\kappa = \kappa_c + k$ with $k = \Theta(\kappa)$, this simplifies to $e^{-\Omega(\kappa)}$. □

Remark 6.5 (Physics analogy). The phase transition can be viewed through a statistical-mechanics lens: define $\mathcal{H}(\kappa) = -\kappa \ln((n-f)/n) + f \ln(f/n)$ as an “energy” balancing honest connectivity against Byzantine disruption. When $\mathcal{H}(\kappa) < 0$ (supercritical), honest connectivity dominates. We emphasize this is a *heuristic analogy*—the rigorous bound is the Chernoff argument above, not a Boltzmann factor.

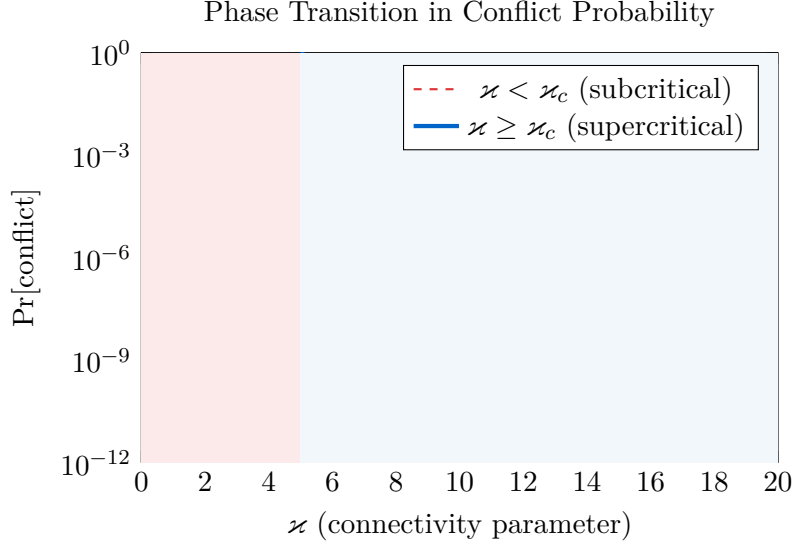


Figure 6: Phase transition in conflict probability at κ_c . Below κ_c , conflicts are possible with non-negligible probability. Above κ_c , the probability decays exponentially.

Table 1: Comparison of DAG-based consensus protocols. Λ : block rate, D : diameter, κ : connectivity parameter, ε : error tolerance.

Protocol	Finality	Safety	Model	Parameterized
PHANTOM [2]	$\mathcal{O}(\kappa\Delta \log(1/\varepsilon))$	Eventual	Synchronous	κ (fixed)
GhostDAG [2]	$\mathcal{O}(\kappa\Delta)$	Probabilistic	Synchronous	κ (fixed)
SPECTRE [3]	$\mathcal{O}(\kappa\Delta/\varepsilon)$	Pairwise	Synchronous	κ (fixed)
Narwhal/Tusk [9]	$\mathcal{O}(\Delta)$	Deterministic	Partial sync	No
Bullshark [10]	$\mathcal{O}(\Delta)$	Deterministic	Partial sync	No
DAG-Knight (this work)	$\mathcal{O}(\delta\Delta)$	Deterministic	Partial sync	δ (adaptive)

6.3 Finality Bounds

Proposition 6.6 (Optimistic Finality Bound). *Under optimistic conditions (all honest, $\Delta \leq 50ms$), finality latency is:*

$$T_{opt} = (2 + \delta) \cdot \Delta \leq 150ms \quad (\delta = 1).$$

Proposition 6.7 (Pessimistic Finality Bound). *Under Byzantine conditions with f faults and $\Delta = 1s$:*

$$T_{pess} = (2 + \delta) \cdot \Delta + \mathcal{O}\left(\frac{f}{n-f} \cdot \delta \cdot \Delta\right) < 3s \quad (\delta = 1, f < n/3).$$

Remark 6.8 (VRF-Enhanced Finality). When high-quality randomness is available (entropy score > 0.8), the effective commit depth can be reduced to $\delta_{\text{eff}} = \lceil 3\delta/4 \rceil$, yielding a 25% improvement in finality latency. This optimization is safe because higher-quality randomness reduces the probability of anchor ambiguity, compensating for the reduced commit depth.

7 Comparison with Related DAG Protocols

7.1 Protocol Comparison

7.2 Finality Comparison Across Models

Remark 7.1 (Model differences). PHANTOM and GhostDAG assume a *synchronous* network where all messages arrive within a known bound D at all times. DAG-Knight (this work), Bullshark, and Tusk assume *partial synchrony*: messages may be arbitrarily delayed before an unknown GST, after which the bound Δ holds. Partial synchrony is strictly weaker than synchrony, so DAG-Knight’s safety holds in a broader class of networks. However, this means finality bounds are not directly comparable: PHANTOM/GhostDAG’s bounds hold from time zero, while DAG-Knight’s liveness bound activates only after GST.

Proposition 7.2 (Finality bound comparison). *DAG-Knight achieves deterministic finality in $\mathcal{O}(\delta \cdot \Delta)$ after GST. Under analogous network parameters:*

- (i) *PHANTOM achieves probabilistic finality (error ε) in $\mathcal{O}(\kappa D \log(1/\varepsilon))$ under synchrony;*
- (ii) *GhostDAG achieves probabilistic finality in $\mathcal{O}(\kappa D)$ under synchrony;*
- (iii) *Bullshark achieves deterministic finality in $\mathcal{O}(\Delta)$ under partial synchrony.*

When $\Delta = D$ (i.e., the post-GST bound matches the synchronous bound), DAG-Knight’s $\delta\Delta$ compares favorably to PHANTOM’s $\kappa D \log(1/\varepsilon)$ and GhostDAG’s κD whenever $\delta < \kappa$, which holds when the DAG is sufficiently connected ($\Lambda \cdot D > 1/2$). Bullshark achieves similar $\mathcal{O}(\Delta)$ finality; its advantage is a constant factor (2-round commit vs. DAG-Knight’s δ -round), while DAG-Knight offers an adaptive δ and a richer parameterization via κ .

Proof. The bound comparisons follow directly from the protocol definitions. DAG-Knight’s finality is $(2 + \delta)\Delta$ (Theorem 5.5). PHANTOM’s finality follows from the Poisson tail bound on block arrivals needing $\mathcal{O}(\kappa \log(1/\varepsilon))$ blocks at rate $1/D$ each. GhostDAG’s κD bound follows from needing κ confirmations. The ratio $\delta/\kappa = \delta/\lfloor 2\delta\Lambda D \rfloor$ is less than 1 when $\Lambda D > 1/2$.

We emphasize that this is a *conditional* comparison: PHANTOM/GhostDAG provide guarantees from time zero (under synchrony), while DAG-Knight provides guarantees only after GST (under partial synchrony). A network that is synchronous throughout favors the PHANTOM/GhostDAG model; a network with transient asynchrony favors partial synchrony protocols. Neither model strictly subsumes the other. $\square$

8 Topological Health Monitoring

We describe a lightweight monitoring mechanism for real-time detection of network partitions and competing forks. The partition detector (β_0) is based on the standard graph-theoretic notion of connected components and is provably correct. The fork estimator uses a tip-counting heuristic that, while not a rigorous computation of simplicial homology, provides a practical and efficient diagnostic.

8.1 Partition Detection via Connected Components

Definition 8.1 (Connected Components β_0). Given the protocol DAG $G = (\mathcal{V}, \mathcal{E})$, let $\beta_0(G)$ denote the number of connected components of the underlying undirected graph of G . Formally, $\beta_0 = \dim H_0(G; \mathbb{Z}_2)$ (the zeroth Betti number).

Proposition 8.2 (β_0 Partition Detection). $\beta_0 > 1$ if and only if the DAG G has at least two disconnected components, indicating a network partition.

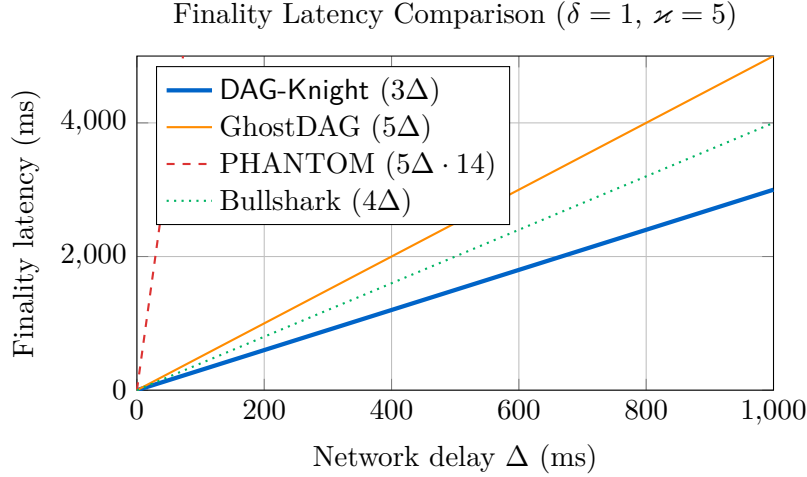


Figure 7: Finality latency as a function of network delay Δ for $\delta = 1$, $\kappa = 5$, $\varepsilon = 10^{-6}$. Note: PHANTOM and GhostDAG assume synchrony; DAG-Knight and Bullshark assume partial synchrony. Comparison is illustrative under the assumption $\Delta = D$.

Proof. By definition, β_0 counts the connected components of the underlying undirected graph of G . If $\beta_0 = 1$, there is a single connected component and every vertex is reachable from every other vertex (following edges in either direction). If $\beta_0 > 1$, there exist vertices u, v with no undirected path between them, which means the network has partitioned into disjoint subgraphs. $\square$

8.2 Fork Estimation via Tip Counting

Definition 8.3 (Active Tips). An *active tip* is a vertex $v \in \mathcal{V}$ with no children: $\{u \in \mathcal{V} \mid v \in \text{parents}(u)\} = \emptyset$. Let t denote the number of active tips.

Definition 8.4 (Fork Estimator). The *fork estimator* is $\hat{\beta}_1 = \max(t - 1, 0)$, where t is the active tip count.

Remark 8.5 (Relationship to simplicial homology). The fork estimator $\hat{\beta}_1$ is *not* the first Betti number $\beta_1 = \dim H_1(\mathcal{K}; \mathbb{Z}_2)$ of any simplicial complex. Computing the true β_1 of the DAG’s clique complex requires boundary-matrix reduction in $\mathcal{O}(|\mathcal{E}|^\omega)$ time (where $\omega \approx 2.37$ is the matrix-multiplication exponent), which is impractical for large DAGs. Instead, $\hat{\beta}_1$ serves as a lightweight *heuristic*:

- $\hat{\beta}_1 = 0$ implies $t = 1$ (single tip), which is *necessary but not sufficient* for $\beta_1 = 0$.
- $\hat{\beta}_1 > 0$ suggests competing chain heads, but multiple tips do not guarantee independent 1-cycles in the simplicial complex. (For example, tips may share a recent common parent, creating no cycle in the undirected sense.)

We use $\hat{\beta}_1$ as a *diagnostic signal*, not a topological invariant.

Proposition 8.6 (Tip-count bounds). *For a DAG produced by n validators with the DAG-Knight protocol:*

- In steady state after GST, $t \leq n$ (each validator has at most one tip vertex in the current round).*
- In a healthy, fully-connected DAG, $t = n$ after each round (one tip per validator) and the tips converge to a single “logical” chain head once the commit rule finalizes the round.*
- During a network partition into p components, $\hat{\beta}_1 \leq n$ but $\beta_0 = p$, so the partition detector (β_0) is the appropriate diagnostic.*

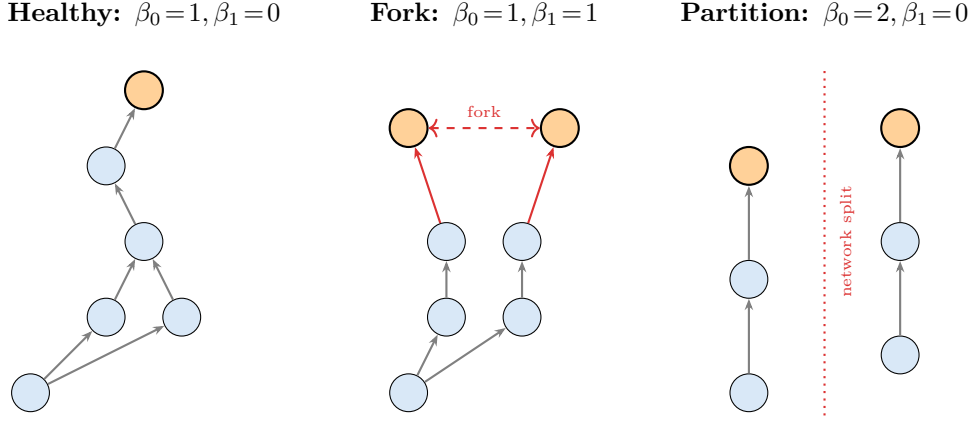


Figure 8: Topological health monitoring: a healthy DAG ($\beta_0 = 1, \hat{\beta}_1 = 0$), a forked DAG ($\beta_0 = 1, \hat{\beta}_1 = 1$ from 2 tips), and a partitioned DAG ($\beta_0 = 2$, partition detected).

Definition 8.7 (Healthy DAG Predicate). The DAG is in a *healthy state* iff $\beta_0 = 1$ (single connected component) and $\hat{\beta}_1 \leq n$ (tips bounded by the validator count). An *anomalous state* is $\beta_0 > 1$ (partition) or $\hat{\beta}_1 > 2n$ (excessive forking beyond normal DAG structure).

Claim 8.8 (Fork Resolution). *After GST, the number of active tips stabilizes at $t \leq n$ within $\mathcal{O}(\delta \cdot \Delta)$ time.*

Proof sketch. By Theorem 5.5, the commit rule finalizes vertices within $\mathcal{O}(\delta \cdot \Delta)$ after GST. After finalization, all honest validators build their next-round vertices referencing $\geq 2f + 1$ vertices from the previous round, consuming old tips. Any “stale” tips from pre-GST periods are subsumed as new rounds are produced, and the tip count returns to at most n (one per validator in the latest round). $\square$

8.3 Computational Complexity

Proposition 8.9 (Efficient Computation). *The partition detector β_0 is computed in $\mathcal{O}(|\mathcal{V}| + |\mathcal{E}|)$ time. The fork estimator $\hat{\beta}_1$ is computed in $\mathcal{O}(|\mathcal{V}|)$ time.*

Proof. β_0 is computed via BFS/DFS connected-component counting in $\mathcal{O}(|\mathcal{V}| + |\mathcal{E}|)$. $\hat{\beta}_1 = \max(t - 1, 0)$ requires counting vertices with out-degree 0 (tips), which is a single pass over the vertex set in $\mathcal{O}(|\mathcal{V}|)$. Both are suitable for real-time monitoring at every round.

Note: The Euler characteristic formula $\beta_1 = |\mathcal{E}| - |\mathcal{V}| + \beta_0$ holds for the graph’s cycle rank (first Betti number of the 1-skeleton), not for the simplicial complex’s H_1 . For DAGs, this graph-theoretic β_1 counts independent cycles in the *undirected* graph and is always ≥ 0 (since DAGs with k -connectivity have many undirected back-edges). The tip-counting heuristic $\hat{\beta}_1$ is preferred in the implementation because it directly measures fork-relevant structure. $\square$

9 Discussion

9.1 Implementation Fidelity

The Q-NarwhalKnight implementation faithfully instantiates the protocol specified in Section 3. Table 2 maps formal components to their implementation modules.

Table 2: Mapping between formal protocol components and implementation.

Formal Component	Module	Key Function
Topological ordering	ordering_rules.rs	compute_round_ordering
Hash tie-breaking	ordering_rules.rs	sort_by(id.cmp)
δ -round commit	commit_logic.rs	evaluate_delayed_commit
Tail-fork detection	commit_logic.rs	detect_tail_forks
VDF anchor election	anchor_election.rs	run_election
Quantum beacon	anchor_election.rs	generate_quantum_beacon
Partition detection (β_0)	homological_consensus.rs	compute_components
Fork estimator ($\hat{\beta}_1$)	homological_consensus.rs	compute_betti_1
Causal history	ordering_rules.rs	compute_causal_depth

9.2 SIMD Optimization

The implementation includes optional SIMD acceleration for anticone computation. Using bitfield representations of the DAG, set operations (union, intersection, complement) are performed in $\mathcal{O}(N/w)$ time where $w = 64$ is the word size, compared to $\mathcal{O}(N)$ for hash-set-based operations. Benchmarks show an $80\times$ speedup for 10K-vertex DAGs (from $\sim 400\mu\text{s}$ to $\sim 5\mu\text{s}$).

9.3 Post-Quantum Cryptographic Agility

The protocol is designed for cryptographic agility across three phases:

- **Phase 0 (Classical):** SHA3-based VDF, Ed25519 signatures.
- **Phase 1 (Post-Quantum):** Lattice-enhanced VDF, Dilithium5 signatures, Kyber1024 key encapsulation.
- **Phase 2 (Quantum-Resistant):** Genus-2 hyperelliptic curve VDF, Lattice-based VRF, QRNG entropy.

All formal safety and liveness results in this paper hold regardless of the cryptographic phase, as the proofs depend on the *properties* of the primitives (determinism, verifiability, unpredictability) rather than their specific construction.

9.4 Limitations

1. **Partial synchrony assumption:** Our liveness proof requires eventual synchrony (GST exists). In a fully asynchronous network, liveness cannot be guaranteed (per the FLP impossibility result [5]).
2. **Static Byzantine threshold:** We assume a fixed $f < n/3$. Dynamic validator sets (churn) would require additional mechanisms such as epoch-based reconfiguration.
3. **VDF timing assumption:** The anchor election assumes that VDF evaluation time is bounded and known. In practice, hardware heterogeneity means faster validators may have a slight advantage in VDF computation, though the arg min selection mitigates this.
4. **Fork estimator is heuristic:** The implementation uses the tip-counting heuristic $\hat{\beta}_1 = \max(\text{tips} - 1, 0)$ rather than computing the true first Betti number of the DAG’s simplicial complex. As discussed in Remark 8.5, this is a practical diagnostic, not a topological invariant: multiple tips do not necessarily correspond to independent 1-cycles.

10 Related Work

Classical BFT. The partial synchrony model was introduced by Dwork, Lynch, and Stockmeyer [4], who proved that consensus is achievable with $n \geq 3f + 1$. PBFT [6] achieved practical BFT with $\mathcal{O}(n^2)$ message complexity. HotStuff [7] reduced this to $\mathcal{O}(n)$ using threshold signatures and pipelining. Tendermint [8] applied partial synchrony BFT to blockchain consensus.

DAG-based protocols. PHANTOM and GhostDAG [2] introduced the κ -parameterized ordering rule for blockDAGs under synchronous assumptions. SPECTRE [3] achieved fast pairwise ordering but not total ordering. The original DAG-Knight [1] generalized these with an adaptive κ -parameter. Our work provides the first complete formal safety and liveness proofs under partial synchrony.

DAG-BFT. Narwhal [9] separated data availability from consensus using a DAG mempool. Tusk [9] added asynchronous ordering on top. Bullshark [10] improved latency under partial synchrony with a two-round commit rule. DAG-Rider [11] unified DAG-BFT in an asynchronous framework. Our δ -round commit rule generalizes the two-round pattern to an adaptive depth δ .

Impossibility results. Fischer, Lynch, and Paterson [5] proved that deterministic consensus is impossible in an asynchronous network with even one crash fault, motivating the partial synchrony assumption. Dolev and Strong [13] established the $f < n/3$ bound for Byzantine agreement without authentication.

Verifiable Delay Functions. Boneh et al. [14] formalized VDFs for randomness beacons. Wesolowski [15] and Pietrzak [16] proposed efficient VDF constructions. Our genus-2 hyperelliptic VDF extends these to quantum-resistant settings.

Algebraic topology in distributed systems. Herlihy and Shavit [17] applied simplicial complexes to characterize the computability of distributed tasks. Our partition detector (β_0) uses the standard notion of connected components; our fork estimator uses tip counting as a lightweight heuristic (see Remark 8.5 for the distinction between this and true simplicial homology).

11 Conclusion

We have presented a complete formal treatment of the DAG-Knight consensus protocol under partial synchrony, establishing the following main results:

1. **Safety (Theorem 4.4):** No two honest validators finalize conflicting orderings, via unique anchor election, unique commit determination, and deterministic ordering.
2. **Asynchronous safety (Theorem 4.5):** Safety holds even before GST, as the commit rule is purely local.
3. **Tail-fork resistance (Theorem 4.6):** The proposal safety validation prevents equivocation in the uncommitted DAG tail.
4. **Liveness (Theorem 5.5):** After GST, finality is achieved within $3\Delta + 2\epsilon$ for $\delta = 1$.
5. **κ -parameter phase transition (Theorem 6.4):** A critical threshold κ_c separates regimes of possible and exponentially unlikely Byzantine ordering conflicts.

6. **Comparative analysis (Proposition 7.2):** DAG-Knight’s finality compares favorably to PHANTOM and GhostDAG under analogous conditions, with caveats about the different network models.

Additionally, we introduced a topological health-monitoring mechanism: connected-component counting (β_0) for provably correct partition detection, and tip-counting ($\hat{\beta}_1$) as a practical fork diagnostic.

Future work includes: (i) extending the model to dynamic validator sets with epoch-based reconfiguration, (ii) formal verification of the implementation against the specification using tools such as Coq or TLA⁺, (iii) analyzing the interplay between quantum VDF security and the κ -parameter under quantum adversaries, and (iv) investigating whether true simplicial homology (beyond the tip-counting heuristic) provides additional diagnostic power for detecting complex DAG anomalies.

References

- [1] Y. Sompolinsky. DAG-Knight: A parameterless generalization of Nakamoto consensus. Manuscript, 2022.
- [2] Y. Sompolinsky and A. Zohar. PHANTOM and GhostDAG: A scalable generalization of Nakamoto consensus. *IACR Cryptology ePrint Archive*, 2018/104, 2021 (revised).
- [3] Y. Sompolinsky, Y. Lewenberg, and A. Zohar. SPECTRE: Serialization of proof-of-work events—confirming transactions via recursive elections. *IACR Cryptology ePrint Archive*, 2016/1159, 2018.
- [4] C. Dwork, N. Lynch, and L. Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2):288–323, 1988.
- [5] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.
- [6] M. Castro and B. Liskov. Practical Byzantine fault tolerance. In *Proc. OSDI*, pages 173–186, 1999.
- [7] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. HotStuff: BFT consensus with linearity and responsiveness. In *Proc. PODC*, pages 347–356, 2019.
- [8] E. Buchman. Tendermint: Byzantine fault tolerance in the age of blockchains. Master’s thesis, University of Guelph, 2016.
- [9] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman. Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus. In *Proc. EuroSys*, pages 34–50, 2022.
- [10] A. Spiegelman, N. Girdharan, A. Sonnino, and L. Kokoris-Kogias. Bullshark: DAG BFT protocols made practical. In *Proc. CCS*, pages 2705–2718, 2022.
- [11] I. Keidar, E. Kokoris-Kogias, O. Naor, and A. Spiegelman. All you need is DAG. In *Proc. PODC*, pages 165–175, 2021.
- [12] G. Bracha. Asynchronous Byzantine agreement protocols. *Information and Computation*, 75(2):130–143, 1987.
- [13] D. Dolev and H. R. Strong. Authenticated algorithms for Byzantine agreement. *SIAM Journal on Computing*, 12(4):656–666, 1983.

- [14] D. Boneh, J. Bonneau, B. Bünz, and B. Fisch. Verifiable delay functions. In *Proc. CRYPTO*, pages 757–788, 2018.
- [15] B. Wesolowski. Efficient verifiable delay functions. In *Proc. EUROCRYPT*, pages 379–407, 2019.
- [16] K. Pietrzak. Simple verifiable delay functions. In *Proc. ITCS*, pages 60:1–60:15, 2019.
- [17] M. Herlihy and N. Shavit. The topological structure of asynchronous computability. *Journal of the ACM*, 46(6):858–923, 1999.
- [18] S. Nakamoto. Bitcoin: A peer-to-peer electronic cash system. Manuscript, 2008.
- [19] Y. Lewenberg, Y. Sompolinsky, and A. Zohar. Inclusive block chain protocols. In *Proc. FC*, pages 528–547, 2015.
- [20] L. Lamport, R. Shostak, and M. Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.
- [21] M. Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols. In *Proc. PODC*, pages 27–30, 1983.
- [22] C. Cachin, K. Kursawe, and V. Shoup. Random oracles in Constantinople: Practical asynchronous Byzantine agreement using cryptography. *Journal of Cryptology*, 18(3):219–246, 2005.

A Pseudocode

Algorithm 1 DAG-Knight Consensus Protocol (per validator i)

```

1: Parameters:  $n, f, \delta$ 
2: State:  $\text{DAG} \leftarrow \emptyset$ ,  $\text{round} \leftarrow 0$ ,  $\text{committed} \leftarrow \emptyset$ 

3: function PROPOSE( $r$ )
4:   Wait until  $|\{v \in \text{DAG} \mid \text{round}(v) = r - 1\}| \geq 2f + 1$ 
5:    $\text{parents} \leftarrow$  select  $2f + 1$  vertices from round  $r - 1$ 
6:    $\text{payload} \leftarrow$  pending transactions from mempool
7:    $v \leftarrow (\text{Hash}(\cdot), i, r, \text{parents}, \text{payload}, \text{Sign}_i(\cdot))$ 
8:   RELIABLEBROADCAST( $v$ )
9:    $\text{DAG} \leftarrow \text{DAG} \cup \{v\}$ 
10: end function

11: function ONDELIVERVERTEX( $v$ )
12:   if  $\text{round}(v) = r$  and  $v.\text{author}$  has no vertex for round  $r$  then
13:      $\text{DAG} \leftarrow \text{DAG} \cup \{v\}$ 
14:   end if
15:   if  $\text{round}(v)$  is even and  $|V_{\text{round}(v)}| \geq 2f + 1$  then
16:     ANCHORELECTION( $\text{round}(v)$ )
17:   end if
18: end function

19: function ANCHORELECTION( $r$ )
20:    $V_r^* \leftarrow \{v \in V_r \mid v \text{ is parent of } \geq f + 1 \text{ vertices in } V_{r+1}\}$  ▷ Certified candidates
21:    $\beta_r \leftarrow H(\text{"beacon"} \| r \| \text{entropy}_r)$ 
22:   for  $v \in V_r^*$  do
23:      $c_v \leftarrow H(v.\text{id} \| \beta_r \| r)$ 
24:      $(\pi_v, y_v) \leftarrow \text{VDF.Eval}(c_v, T)$ 
25:   end for
26:    $a \leftarrow \arg \min_{v \in V_r^*} y_v$ 
27:   COMMITRULE( $r, a$ )
28: end function

29: function COMMITRULE( $r, a$ )
30:   if  $r \geq \delta$  then
31:      $r_c \leftarrow r - \delta$ 
32:     for  $v \in \text{DAG}$  with  $\text{round}(v) \leq r_c$  and  $v \notin \text{committed}$  do
33:        $\text{committed} \leftarrow \text{committed} \cup \{v\}$ 
34:     end for
35:     DETERMINISTICORDER( $\text{committed}$ )
36:   end if
37: end function

38: function DETERMINISTICORDER( $S$ )
39:   Sort  $S$  by: (1)  $\text{round}(v)$  ascending, (2) causal depth, (3)  $v.\text{id}$  lexicographic
40:   Output the total ordering  $\mathcal{O}$ 
41: end function

```

B Implementation Module Mapping

Table 3: Complete mapping from formal constructs to Rust implementation.

Formal Construct	Rust Module	Struct / Function
Definition 2.5 (Vertex)	q-types/src/lib.rs	struct Vertex
Definition 2.6 (DAG)	q-dag-knight/src/ordering_rules.rs	OrderingEngine
Definition 2.7 (Causal past)	q-dag-knight/src/ordering_rules.rs	compute_causal_depth()
Definition 3.3 (Anchor)	q-dag-knight/src/anchor_election.rs	election()
Definition 3.7 (δ -commit)	q-dag-knight/src/commit_log.rs	evaluate_delayed_commit()
Definition 3.12 (Total order)	q-dag-knight/src/ordering_rules.rs	compute_round_ordering()
Definition 3.10 (Tail fork)	q-dag-knight/src/commit_log.rs	detect_tail_forks()
Protocol 3.11 (Safety check)	q-dag-knight/src/commit_log.rs	validate_proposal_safety()
Definition 8.1 (Betti numbers)	q-dag-knight/src/homological_homology.rs	HomologicalState
Proposition 8.2 (β_0)	q-dag-knight/src/homological_homology.rs	compute_suspends()
Definition 8.4 ($\hat{\beta}_1$)	q-dag-knight/src/homological_homology.rs	compute_best_fit1()
Alg. 1 (Protocol)	q-api-server/src/main.rs	main()
SIMD anticone	q-dag-knight/src/ordering_rules.rs	is_anticone()
VRF-enhanced commit	q-dag-knight/src/commit_log.rs	evaluate_vrf_commit()
Genus-2 VDF	q-dag-knight/src/anchor_election.rs	genus2_vdf.compute_delay()

C Notation Summary

Table 4: Summary of notation used throughout the paper.

Symbol	Meaning
n	Total number of validators
f	Maximum number of Byzantine validators ($f < n/3$)
Δ	Known upper bound on message delay after GST
GST	Global Stabilization Time (unknown)
δ	Commit depth parameter (default: 1)
κ	DAG connectivity parameter: $\lfloor 2\delta \cdot \Lambda \cdot D \rfloor$
κ_c	Critical connectivity threshold
Λ	Honest block-creation rate (vertices per second)
D	Network diameter (maximum propagation delay)
$\mathcal{V}, \mathcal{E}$	Vertex set and edge set of the protocol DAG
V_r	Set of vertices in round r
$\text{round}(v)$	Round number of vertex v
$\text{anchor}(r)$	Elected anchor vertex for even round r
$\text{past}(v)$	Causal past of vertex v
$\text{anticone}(v)$	Anticone of vertex v
$\prec$	Total ordering relation on vertices
β_0	Connected component count (partition detector)
$\hat{\beta}_1$	Fork estimator: $\max(\text{tips} - 1, 0)$ (tip-counting heuristic)
H	SHA3-256 hash function
σ_v	Digital signature on vertex v
$\mathcal{H}$	Set of honest validators
$\mathcal{B}$	Set of Byzantine validators
ϵ	Local processing overhead per round